Documento 04 · Soluciones y errores frecuentes

Para: instructor · DeepSeek Harness (dsh) · Generado el 30/10/2026

Soluciones y errores frecuentes

Documento de apoyo para el instructor del seminario **"Del chat al agente: qué es un harness y cómo se usa"**.

Herramienta: **opencode**, un harness de agentes de código abierto, con modelos gratuitos y sin clave de API. Referencia verificada: opencode 1.19.5 (30 de octubre de 2026). Cambia muy rápido (versiones casi semanales) y los modelos gratuitos son «por tiempo limitado»: si algo no coincide con lo que ves, manda lo que muestra la pantalla. Audiencia: profesionales que ya usan la IA como chat o bot. No se requiere experiencia previa en programación. Fuente de verdad de lo que ocurre en cada práctica: el cuaderno del alumno `03_laboratorio.md`. Este documento debe coincidir con él.

Cómo usar este documento

- Tenlo abierto en una segunda ventana mientras la clase practica.
- Localiza la práctica en la que esté el grupo y busca en la tabla la fila que coincida con lo que ves en su pantalla.
- Las señales de "salió bien" sirven para confirmar en voz alta que el alumno va por buen camino: dilo, refuerza.
- Si el problema aparece en varios portátiles a la vez, ve directamente a **Problemas técnicos de sala**.
- Si te quedas sin palabras delante de la clase, usa las **Frases de rescate**.
- Recuerda las dos ideas que se repiten: **la carpeta de trabajo es la sala del agente** y **el permiso es el "¿seguro?"**. La carpeta reduce el riesgo, pero opencode **no es un sandbox**.
- Si algún alumno usa el carril B (Claude Code o Codex, descrito en `carriles_alumno.md`), su comportamiento de serie difiere del de opencode: se indica en una línea en las prácticas 1 a 4.

Recordatorio de arranque

- Instalación (una sola vez): `npm install -g opencode-ai`. Se comprueba con `opencode --version`.
- Arranque: entrar en la carpeta de práctica con `cd` y escribir `opencode web`.
- Se abre el navegador solo; si no, copiar la dirección que aparece en la terminal (por ejemplo, **http://127.0.0.1:4096**). El puerto puede variar; se fija con `--port`.
- **Ctrl + C** apaga opencode; cerrar la terminal también lo apaga.
- No hay clave de API que configurar. Si PowerShell bloquea la ejecución de scripts, usar **cmd**.

Práctica 1 — Primer arranque y primer encargo

Objetivo: arrancar el harness, abrir la carpeta como proyecto, comprobar el modelo gratuito y conseguir que el agente lea una carpeta propia y haga un índice.

Qué debería pasar

- El alumno comprueba `opencode --version`, entra en `practica_agente` y lanza `opencode web`; se abre la interfaz web en el navegador, en español.
- Añade su carpeta con **Añadir proyecto** (escribiendo `Documents/practica_agente`) y pulsa **Nueva sesión**.
- Debajo del cuadro de mensajes ve el modelo (por ejemplo, Big Pickle) y, en **Seleccionar modelo**, elige uno marcado como **Gratis**: GLM-5.3-Flash o DeepSeek V4.1 Flash.
- Escribe el primer encargo y el agente lee archivos reales de `informes` y crea `INDICE.md`.

Cómo saber que salió bien

- La interfaz carga y se ve la sesión del proyecto `practica_agente`.
- El agente **no responde de golpe**: va dando pasos visibles (*Explorado*, *Shell*, *Escribir*…).
- Existe `INDICE.md` y el agente dice dónde lo ha dejado (puede haberlo guardado dentro de `informes`: el encargo no decía dónde).
- El índice menciona **nombres de archivos reales** de la carpeta: ha leído, no ha inventado. Los originales siguen intactos.
- El alumno sabe decir qué herramientas ha usado el agente (mirar la carpeta, leer, ejecutar una orden, escribir).

Solución de referencia

Petición del alumno:

Mira qué hay en la carpeta `informes` y hazme un índice de los archivos, agrupados por tipo, en un archivo nuevo llamado `INDICE.md`. No modifiques ningún archivo original.

Qué haría el agente:

1. Explora el contenido de la carpeta `informes` (paso *Explorado*).
2. Lee o lista los archivos; puede usar una orden de terminal (paso *Shell*).
3. Escribe `INDICE.md` (paso *Escribir*) con los archivos agrupados por tipo.
4. Dice dónde ha dejado el archivo nuevo y confirma que no ha tocado los originales.
5. Si algo queda fuera de la carpeta del proyecto, pide permiso antes de acceder.

Carril B (Claude Code o Codex): se arranca con `claude` o `codex` desde la carpeta, tras iniciar sesión con la cuenta del alumno; no hay que elegir modelo gratuito. Claude Code pregunta antes de editar de serie, así que verá avisos que con opencode no aparecen.

Errores frecuentes

Lo que se ve	Qué está pasando	Qué hacer
«opencode no se reconoce como una orden»	Terminal abierta antes de instalar, o instalación incompleta (falta Node.js u opencode)	Cerrar y abrir la terminal; si sigue, repetir la instalación de la preparación previa
PowerShell se niega a ejecutar opencode (scripts bloqueados)	La política de ejecución de scripts de PowerShell	Usar cmd en lugar de PowerShell
La página no carga	La terminal se ha cerrado, o la dirección es otra	Comprobar que la terminal sigue abierta y copiar exactamente la dirección que muestra (el puerto puede no ser el 4096)
El alumno escribe en la terminal y no pasa nada	La terminal solo arranca el servidor; el trabajo se hace en el navegador	Explicar que la terminal es el motor encendido y el navegador es el volante
No encuentra su carpeta en Añadir proyecto	El buscador no muestra carpetas ocultas o temporales	Escribir la ruta desde el usuario: Documents/practica_agente
El agente ha trabajado en una carpeta distinta de la esperada	Se abrió otro proyecto, o se lanzó opencode web desde otra carpeta	Comprobar el proyecto de la sesión y abrir la carpeta correcta antes de pedir nada
Error del modelo, respuesta vacía o «saturado»	Los modelos gratuitos a veces se saturan	Pulsar el nombre del modelo, elegir otro marcado como Gratis y repetir
El agente contesta en inglés o de forma muy genérica	No ha leído nada, solo ha conversado; o no se le pidió el idioma	Pedir explícitamente "en español" y "mira los archivos de la carpeta"
El agente tarda mucho	Está trabajando	Mirar los pasos: si avanzan, dejarlo
El alumno pega datos reales en la carpeta	Cree que gratis es privado	Recordar que lo que el agente lee se envía al proveedor del modelo; GLM-5.3-Flash y DeepSeek V4.1 Flash no guardan datos, Big Pickle y otros pueden usarlos. Gratis no es privado

Plan B si no sale

- Si no arranca nada, comprueba Node.js y vuelve a lanzar el comando en una terminal nueva (o en cmd).
- Si no consigue instalar opencode, pásale a un compañero que ya lo tenga funcionando y siga la práctica por parejas.
- Si todos los gratuitos están saturados, que espere un minuto y pruebe otro modelo.
- Mientras se arregla, pídele que escriba en papel qué carpeta quiere resumir y qué le va a pedir al agente.

Si alguien va muy rápido

- Que pida el índice agrupado también por subcarpeta, o que pida un resumen por tipo de archivo.
- Que pida al agente que, a partir de los documentos, proponga una lista de tareas pendientes.

Práctica 2 — Provocar una aprobación

Objetivo: entender qué es la carpeta de trabajo, qué queda fuera de ella y qué significa el aviso de permiso.

Qué debería pasar

- El alumno, en una sesión del proyecto `practica_agente`, pide al agente crear un archivo **fuera** de la carpeta (por ejemplo, en el escritorio).
- La operación no se ejecuta sola: aparece el aviso **«Permiso requerido»**: *Acceder a archivos fuera del directorio del proyecto*, con la ruta debajo.
- El aviso tiene tres botones: **Denegar**, **Permitir siempre** y **Permitir una vez**. El agente se queda esperando.
- El alumno lee el aviso, responde por escrito a las tres preguntas (qué pide, qué pasaría si digo que sí, puedo acotarlo) y pulsa **Permitir una vez**.
- En la segunda petición (`prueba_agente2.txt`) vuelve a preguntar; el alumno pulsa **Denegar** y el agente se **para**, sin intentarlo por otro camino.

Cómo saber que salió bien

- Aparece el aviso «Permiso requerido» y el agente **se detiene** en vez de actuar.
- El aviso indica qué operación quiere hacer y sobre qué ruta.
- El alumno ha leído el aviso **antes** de pulsar nada y sabe explicar con sus palabras qué le pedía.
- Con **Permitir una vez** la operación se completa; al repetir la petición **vuelve a preguntar**; al **Denegar**, el agente se detiene.
- El alumno ha borrado los archivos de prueba del escritorio.

Solución de referencia

Petición del alumno:

Crea un archivo llamado `prueba_agente.txt` en el escritorio de mi usuario, con el texto «prueba».

Qué haría el agente:

1. Detecta que el escritorio está **fuera** de la carpeta del proyecto.
2. Muestra el aviso «Permiso requerido» con la ruta exacta a la que quiere acceder.
3. Espera. No escribe nada hasta que el alumno decide.
4. Con **Permitir una vez**, crea el archivo y termina. Con **Denegar**, se para.

Respuestas esperadas a las tres preguntas: (1) escribir un archivo nuevo en el escritorio; (2) aparecería `prueba_agente.txt` en el escritorio y no cambiaría nada más; (3) sí, pidiéndole que lo haga dentro de la carpeta de trabajo.

Carril B (Claude Code o Codex): el aviso tiene otro aspecto, pero las opciones equivalen (sí esta vez, sí a partir de ahora, no). Claude Code debe estar en modo normal, no en Auto; Codex, en su modo de serie, que pide permiso para salir de la carpeta.

Errores frecuentes

Lo que se ve	Qué está pasando	Qué hacer
El agente crea el archivo sin preguntar nada	La ruta está dentro del proyecto, o la sesión pertenece a otra carpeta (por ejemplo, el escritorio ya es el proyecto)	Comprobar el proyecto de la sesión y elegir una ruta claramente externa (escritorio, carpeta personal)

Lo que se ve	Qué está pasando	Qué hacer
No aparece ningún aviso y el archivo se crea igualmente	El agente aprueba solo: por ejemplo, Claude Code en modo Auto, donde otra IA decide en lugar del alumno	Convertirlo en la lección: sin aviso no hay «leer antes de aprobar». Cambiar a modo normal (Shift+Tab) y repetir
El alumno aprueba a ciegas sin leer el aviso	No ha entendido la práctica	Parar y leer el aviso en voz alta: qué operación, qué ruta, qué riesgo
El alumno pulsa Permitir siempre	Quiere que deje de preguntar	Explicar que el permiso deja de proteger: es el mal hábito de quitar el freno porque hace ruido. Repetir la práctica con Permitir una vez
Tras Denegar, el agente se queda parado y el alumno cree que se ha roto	Es lo esperado: al denegar el agente se para	Explicarlo: no lo intenta por otro camino. Para continuar hay que pedirle otra cosa o dentro de la carpeta
Aparecen varios avisos seguidos y el alumno se agobia	El encargo toca varias cosas de golpe	Recordar que «Permitir una vez» es solo esa vez; simplificar la petición a una sola acción
Después de permitir, la operación falla igualmente	Permisos del sistema operativo o antivirus bloqueando esa carpeta	Probar con otra carpeta del usuario y revisar el antivirus

Plan B si no sale

- Si el agente no pide permiso nunca, comprueba que el destino está realmente fuera del proyecto y repite con otra ruta externa.
- Si el aviso no aparece porque el encargo es ambiguo, escribe tú la petición en la pizarra y que la copien.
- Aprovecha para la explicación teórica: el aviso incómodo es el objetivo, no un fallo.

Si alguien va muy rápido

- Que **deniegue** el permiso y observe cómo reacciona el agente. Es la mitad de la lección y casi nadie la hace.
- Que intente lo mismo con dos rutas distintas y compare qué aviso aparece en cada caso.

Práctica 3 — Cambiar de política

Objetivo: comprobar que dentro de la carpeta de trabajo opencode no pregunta de serie, y que el alumno puede cambiar la política con un `opencode.json` para que pregunte también ahí.

Qué debería pasar

- El alumno pide crear `prueba_dentro.txt` **dentro** de la carpeta de trabajo: el agente lo hace **sin aviso** (política de serie: dentro lee, escribe y ejecuta órdenes sin preguntar; solo pregunta al salir).
- Le pide al agente crear `opencode.json` en la raíz con `{"$schema": "https://opencode.ai/config.json", "permission": {"edit": "ask", "bash": "ask"}}`.
- La política se lee al arrancar: el alumno **reinicia** (Ctrl + C, `opencode web`, **Nueva sesión**).
- Al pedir `prueba_dentro2.txt`, ahora **sí** pregunta: *Modificar archivos mediante ediciones, escrituras y parches*. El alumno pulsa **Permitir una vez**.
- Al final borra `opencode.json` y reinicia, o lo deja si prefiere seguir aprobando.

Cómo saber que salió bien

- `prueba_dentro.txt` se crea **sin** aviso de permiso.
- Con `opencode.json` y tras reiniciar, `prueba_dentro2.txt` **sí** pide permiso.
- El alumno explica la diferencia entre la carpeta (dónde trabaja) y la política (qué puede hacer sin preguntar) en una frase.
- Sabe que sin reiniciar el cambio no se nota.

Solución de referencia

Peticiones del alumno:

Crea un archivo llamado `prueba_dentro.txt` en la carpeta de trabajo, con el texto «prueba».

Crea en la raíz de esta carpeta un archivo `opencode.json` con exactamente este contenido: `{"$schema": "https://opencode.ai/config.json", "permission": {"edit": "ask", "bash": "ask"}}`

Crea un archivo llamado `prueba_dentro2.txt` en la carpeta de trabajo, con el texto «prueba».

Qué haría el agente:

1. Primera petición: crea el archivo sin pedir permiso.
2. Segunda petición: escribe `opencode.json` en la raíz (todavía sin preguntar, porque la política no se ha leído).
3. Tercera petición, tras reiniciar y abrir una sesión nueva: muestra el aviso de permiso por editar y espera.
4. Con **Permitir una vez**, crea el archivo.

La conclusión a sacar: **el agente no es más listo ni más tonto; se han cambiado sus reglas.** Hay dos palancas: dónde trabaja (la carpeta) y qué puede hacer sin preguntar (la política).

Carril B (Claude Code o Codex): la política de serie es distinta, y puede que los resultados sean los contrarios. Claude Code pregunta de serie antes de editar, también dentro de la carpeta; con `Shift+Tab` hasta *accept edits* deja de preguntar, y su política fija vive en `.claude/settings.json`. En Codex se usa `/permissions` para elegir un modo más estricto.

Errores frecuentes

Lo que se ve	Qué está pasando	Qué hacer
Tras crear opencode.json, sigue sin preguntar	No se ha reiniciado: la configuración se lee al arrancar. Es el error típico	Ctrl + C en la terminal, opencode web de nuevo y Nueva sesión
Sigue sin preguntar aun reiniciando	opencode.json no está en la raíz del proyecto, o el contenido tiene un error (comillas, llaves)	Pedir al agente la ruta completa del archivo y comparar el contenido con el de la práctica, carácter a carácter
El alumno cierra la terminal en vez de reiniciar y no encuentra opencode	Cerrar la terminal apaga opencode	Abrir una terminal, entrar en la carpeta con cd y volver a lanzar opencode web
Sigue pidiendo permiso aunque la ruta sea interna	La carpeta no es la que se cree, o se ha dejado opencode.json de antes, o hay un acceso directo a otra ubicación	Comprobar el proyecto de la sesión, revisar si existe opencode.json y evitar enlaces, accesos directos y unidades de red
El alumno no encuentra el archivo creado	Lo ha creado en otra subcarpeta de la carpeta de trabajo	Pedir al agente la ruta completa del archivo y abrirla
Pregunta por editar y por ejecutar órdenes	edit y bash son acciones distintas, cada una con su aviso	Explicar que cada tipo de acción se evalúa por separado
Aparecen demasiados avisos en las prácticas siguientes	Se dejó opencode.json puesto	Borrarlo y reiniciar, o aceptar los avisos de más
El alumno cree que la carpeta lo protege de todo	Confunde reducir el riesgo con aislar	Recordar que opencode no es un sandbox: dentro de la carpeta, con la política de serie, puede ejecutar órdenes sin preguntar

Plan B si no sale

- Vuelve a la Práctica 2 y compara las dos pantallas, una al lado de la otra.
- Si la carpeta de trabajo es una unidad de red o una carpeta sincronizada, cambia a una carpeta local sencilla.
- Si el alumno no consigue que el agente cree `opencode.json`, créalo tú con él con un editor de texto y repasa el reinicio.
- Si el grupo va perdido, escribe en la pizarra las tres ideas: dentro, fuera, aviso.

Si alguien va muy rápido

- Que pruebe a poner solo `"edit": "ask"` (sin `bash`), reinicie y anote qué cambia.
- Aprovecha para introducir el **mínimo privilegio**: la política más abierta no es la mejor, es la más arriesgada.

Práctica 4 — Enseñarle un procedimiento

Objetivo: dejar instrucciones permanentes (notas en el **AGENTS.md** o una **skill** guardada) y comprobar que el agente las respeta en una **sesión nueva**.

Qué debería pasar

- El alumno pide al agente que cree `AGENTS.md` en la raíz de la carpeta de trabajo con sus instrucciones (en español; informes en `informes` y sin archivos si solo se responde en el chat; no modificar originales; decir qué archivos se han tocado).
- Abre una **conversación nueva**: en la misma conversación no demuestra nada.
- Pide un resumen de `informes` y el agente cumple las notas sin que se las repitan.
- Opcional: pide la skill `resumen-informes`, **reinicia** opencode y, en una sesión nueva, pide «hazme el resumen de informes».

Cómo saber que salió bien

- `AGENTS.md` existe en la raíz del proyecto (el global estaría en `C:\Users\<tu_usuario>\.config\opencode\AGENTS.md`).
- En una sesión nueva, el agente responde en español, guarda el resultado en `informes` si crea un informe y dice qué archivos ha tocado, **sin recordatorio**.
- Si se usa la skill, existe `.opencode/skills/resumen-informes/SKILL.md` dentro del proyecto y, tras reiniciar, el agente crea `resumen_general.md` con una tabla de tres columnas (archivo, tema, conclusión) sin modificar los originales.
- El alumno explica la diferencia entre lo que dura una conversación y lo que es permanente.

Solución de referencia

Petición del alumno:

Crea en la raíz de esta carpeta de trabajo un archivo `AGENTS.md` con estas instrucciones para ti:

- Responde siempre en español.
- Si creas un archivo de informe, guárdalo en la carpeta `informes`. Si solo respondes en el chat, no crees archivos.
- Nunca modifiques ni borres los archivos originales: si necesitas cambiarlos, crea una copia.
- Al terminar cualquier tarea, dime qué archivos has tocado.

Qué haría el agente:

1. Escribe `AGENTS.md` en la raíz del proyecto con esas líneas: son "las notas que se dejan al becario antes de que empiece".
2. En una **sesión nueva**, ante «Hazme un resumen de los archivos de la carpeta `informes`», responde en español, aplica las reglas y termina diciendo qué archivos ha tocado.
3. Para la skill: ante «Crea una skill llamada `resumen-informes`…», la guarda en `.opencode/skills/resumen-informes/SKILL.md`. **Las skills se cargan al arrancar**: tras reiniciar y abrir sesión nueva, «hazme el resumen de informes» crea `resumen_general.md` con la tabla.
4. Si algo no encaja, lo dice y propone corregir la instrucción, no improvisar cada vez.

Carril B (Claude Code o Codex): Claude Code lee `CLAUDE.md`, **no** `AGENTS.md`; hay que crear un `CLAUDE.md` con la línea `@AGENTS.md`. Codex lee `AGENTS.md` directamente. Las skills van en `.claude/skills/<nombre>/SKILL.md` (Claude Code) o `.agents/skills/<nombre>/SKILL.md` (Codex); el formato del `SKILL.md` es el mismo.

Errores frecuentes

Lo que se ve	Qué está pasando	Qué hacer
En la sesión nueva el agente ignora las instrucciones	AGENTS.md no está en la raíz del proyecto activo (o está en otra carpeta), o no se llegó a crear	Comprobar que el archivo existe en la raíz del proyecto abierto; el global es C:\Users\<tu_usuario>\.config\opencode\AGENTS.md

Lo que se ve	Qué está pasando	Qué hacer
El alumno cree que el agente "recuerda" la conversación anterior	Lo está probando en la misma conversación, o no distingue contexto y notas	Abrir una Nueva sesión: lo permanente vive en AGENTS.md o en la skill, no en la conversación
Instrucciones contradictorias y resultado raro	Hay reglas viejas que chocan con las nuevas	Leer el archivo y limpiar: menos reglas y más claras
La skill no se usa: el agente responde en el chat	No se reinició opencode: las skills se cargan al arrancar. Es el error típico	Ctrl + C, opencode web, sesión nueva y repetir «hazme el resumen de informes»
La skill no se usa ni reiniciando	Ruta o nombre incorrectos: debe estar en .opencode/skills/<nombre>/SKILL.md dentro del proyecto	Pedir al agente la ruta completa y comparar con el ejemplo del repositorio: practica_ejemplo/configuracion_agente/.opencode/skills/
El agente responde en el chat y no guarda nada en informes, aunque la nota lo pida	La nota era ambigua: «cuando crees un informe» no dice si un resumen en el chat cuenta	Reescribir la nota diciendo cuándo se aplica: «si creas un archivo de informe, guárdalo en informes; si solo respondes en el chat, no crees archivos»
El agente guarda las notas en un sitio raro	No se le indicó la ruta y eligió una carpeta del proyecto	Indicar explícitamente «en la raíz de esta carpeta de trabajo»
Un alumno de Claude Code ve que no respeta AGENTS.md	Claude Code lee CLAUDE.md, no AGENTS.md	Crear CLAUDE.md con solo @AGENTS.md

Plan B si no sale

- Si la creación de `AGENTS.md` se atasca, que dicte las instrucciones en voz alta y las guarde a mano en un archivo de texto llamado `AGENTS.md` en la raíz de la carpeta.
- Si no se atreve con skills, quédate con `AGENTS.md`: cubre el mismo objetivo.
- Si el grupo no tiene proyecto propio, usa la carpeta de ejemplo del seminario.

Si alguien va muy rápido

- Que convierta un procedimiento largo ("cómo preparo el informe semanal") en una skill con nombre propio y la pruebe tras reiniciar, en una sesión nueva desde cero.
- Que escriba dos instrucciones y compruebe cuál gana cuando se contradicen: enseña a priorizar.

Práctica 5 — Encargo integrador

Objetivo: realizar un encargo real de principio a fin, con **varias acciones encadenadas**, y verificar el resultado por parte del alumno.

Qué debería pasar

- El alumno redacta el encargo con las cuatro partes: qué quiero, dónde, qué NO tocar y cómo sabré que está bien.
- El agente encadena las acciones (lee, crea, escribe) y los pasos son visibles; pide permiso solo si sale de la carpeta (o si el alumno dejó una política más estricta).
- Al terminar, el agente resume qué ha leído y qué ha escrito, con rutas.
- El alumno **abre los archivos y comprueba** que el contenido se corresponde con lo pedido.

Cómo saber que salió bien

- El agente ha hecho **más de una acción** sin que se lo repitan paso a paso.
- El alumno ha mirado los pasos mientras el agente trabajaba, y no solo el resultado.
- El alumno abre el archivo generado, **cuenta** (elementos, filas) y el contenido encaja con lo pedido.

- Ha comprobado que los archivos que no debía tocar siguen intactos y que la respuesta del agente a «¿qué archivos has tocado?» cuadra con lo que ve.
- Si hubo avisos de permiso, el alumno sabe decir por qué aparecieron.

Solución de referencia

Petición del alumno (ejemplo del cuaderno):

En la carpeta `practica_agente/informes` hay cinco informes en texto. Léelos y créame `resumen_general.md` con una tabla de tres columnas: tema, conclusión y nivel de urgencia. **No modifiques ninguno de los cinco archivos originales.** Al terminar, dime qué archivos has leído y cuántas filas tiene la tabla.

Qué haría el agente:

1. Explora y lee los cinco informes.
2. Escribe `resumen_general.md` con la tabla pedida (cinco filas).
3. Devuelve un resumen: archivos leídos, archivo creado, ruta completa, número de filas y qué partes son interpretación suya (por ejemplo, el nivel de urgencia).
4. Verificación del alumno: abrir el archivo, contar las filas, comprobar que los cinco originales no han cambiado y contrastar la respuesta del agente.

Recuerda a la clase: **el agente no firma su trabajo; lo firma el alumno** al decir que está bien. Y el resultado llega bien escrito y con tono seguro aunque pueda ser incorrecto.

Errores frecuentes

Lo que se ve	Qué está pasando	Qué hacer
El alumno acepta el resultado sin abrir nada	**Confunde "el agente dice que lo ha hecho" con "está hecho"**	**Regla de la sala: el que revisa es el humano; abrir siempre el archivo y contar**
El informe incluye datos que no existen	**El agente ha rellenado huecos; hay que verificar cada afirmación frente a la fuente**	**Pedir que cite de qué archivo sale cada punto, corregir y repetir**
El encargo no avanza y el agente da vueltas	**El encargo es demasiado amplio o ambiguo para una sola petición**	**Partirlo en dos encargos y decir formato, extensión y carpeta de destino**
Se disparan avisos de permiso en cadena	**El encargo sale de la carpeta (escritorio, carpeta personal, unidad de red), o hay un opencode.json restrictivo olvidado**	**Mover el trabajo dentro de la carpeta o borrar opencode.json y reiniciar: la solución no es aprobar cien avisos**
El agente ha modificado un original	**El «qué NO tocar» no estaba dicho, o se aprobó sin mirar**	**Restaurar desde el ZIP y repasar la parte 3 del encargo; la política de serie no protege los archivos de dentro de la carpeta**
El modelo gratuito se satura a mitad del encargo	**Los gratuitos a veces se saturan**	**Elegir otro modelo marcado como Gratis y repetir**
El resultado empeora hacia el final	**La mesa de trabajo (el contexto) se ha llenado de papeles**	**Empezar una sesión nueva y dejar por escrito lo esencial en AGENTS.md o en una skill**

Plan B si no sale

- Reduce el encargo a dos pasos y déjalo ahí: es mejor terminar algo pequeño que no terminar nada.
- Si el agente se pierde, que el alumno pida primero el índice y luego el contenido.
- Si no hay tiempo, hazlo tú en pantalla grande mientras ellos dictan las instrucciones.

Si alguien va muy rápido

- Que añada un paso de verificación: pedir al agente que revise su propio informe y señale sus tres puntos más débiles.
- Después, contrastar esa autocrítica con el archivo real: enseña que el agente se equivoca y se corrige.

Práctica 6 — Datos y modelo (avanzada, opcional)

Objetivo: que el alumno vea que en análisis de datos el resultado es un **programa** que hay que revisar, y que un modelo señala anomalías pero no explica causas.

Las dos trampas del CSV (`lecturas_energia_diarias.csv`, 28 filas, del 1 al 28 de mayo de 2025)

- **Lectura en otras unidades:** el 10 de mayo, `kwh_total` vale `0.751`. Está en MWh; debería ser 751 kWh. Si nadie la corrige, el total sale **31.649,751 kWh** en vez de **32.400 kWh**, y la semana 2 parece consumir 750 kWh menos.
- **Valor vacío:** el 17 de mayo no tiene `horas_operacion`. No afecta al modelo pedido (kWh frente a unidades), pero el agente debe decir qué ha hecho con él.

Valores de referencia (con la lectura corregida)

Semana	kWh totales	kWh nocturnos	Media nocturna diaria
1	7.800	708	~101
2	8.100	720	~103
3	8.500	1.132	~162
4	8.000	733	~105

- Totales: **32.400 kWh** y **17.820 unidades**: 1,82 kWh por unidad, como dice el informe.
- Recta ajustada con los 28 días: unos 289 kWh fijos al día más ~1,37 kWh por unidad. Ajustada sin la semana 3, apenas cambia (~267 + 1,38 × unidades).
- Con cualquiera de los dos ajustes, la semana 3 es la que más consume por encima de lo que le corresponde (unos +295 kWh en la semana con la recta de los 28 días; unos +390 si se ajusta sin la semana 3), y la columna `kwh_nocturno`, que no aparece en el informe, lo confirma: unos 60 kWh más cada noche, también el domingo sin producción.

Cómo saber que salió bien

- El agente describe los datos (filas, columnas, unidades, anomalías) **antes** de calcular.
- El CSV original no se ha modificado; la corrección está en el script y explicada.
- El total cuadra con el informe y el alumno lo ha comprobado él mismo.
- El alumno distingue «la semana 3 se sale del modelo» (dato) de «es el compresor» (hipótesis).

Errores frecuentes

Lo que se ve	Qué está pasando	Qué hacer
El total sale unos 31.650 kWh y nadie lo nota	La lectura en MWh se ha sumado tal cual	Es el momento estrella de la práctica: totales de control contra el informe
El agente «limpia» la fila del 10 de mayo borrándola	Tratar lo raro como basura en lugar de entenderlo	Pedir que corrija y justifique en vez de descartar; comparar los totales
El agente edita el CSV original	El encargo no se ha leído entero o se ha aprobado sin mirar	Restaurar desde el ZIP; repasar la parte «qué NO tocar»
No hay Python o faltan librerías	El agente propone instalar Python o librerías (pandas, matplotlib)	No aceptar instalaciones sin pensarlo; pedirle que use JavaScript con Node.js, que ya está instalado, o que lo haga sin librerías
El informe final afirma que la causa es el compresor	El modelo ha pasado de señal a conclusión	Preguntar «¿qué dato lo demuestra?» y separar hecho de hipótesis

Para el debate (si hay tiempo)

- ¿Qué pasaría si el modelo se usara para predecir un día de 1.200 unidades? (Extrapolación: fuera del rango de los datos.)

- ¿Se podría usar `kwh_nocturno` para predecir el consumo total del mismo día? (Fuga de datos: forma parte de lo que se quiere predecir.)

Problemas técnicos de sala

Síntoma	Causa probable	Solución
Al arrancar dice que el puerto está ocupado, o la dirección no es la 4096	Ya hay otra instancia de opencode u otro programa usando ese puerto; el puerto puede variar	Cerrar la otra instancia, o fijar otro puerto con --port, y copiar la dirección que muestra la terminal
opencode no se reconoce	opencode no está instalado, o la terminal se abrió antes de instalarlo	Instalar con npm install -g opencode-ai, abrir una terminal nueva y comprobar con opencode --version
node o npm no se reconocen, o dan errores de versión	Node.js no está instalado o es demasiado antiguo	Instalar o actualizar Node.js y abrir una terminal nueva para que tome el PATH
PowerShell bloquea la ejecución de scripts	Política de ejecución de PowerShell	Usar cmd
La interfaz no carga en el navegador	Se abrió una dirección equivocada, o la terminal se cerró	Copiar a mano la dirección que aparece en la terminal; probar con otro navegador
El navegador no se abre solo	Puede pasar en algunos equipos	Copiar la dirección de la terminal (por ejemplo, http://127.0.0.1:4096) y pegarla en el navegador
Error de modelo, respuesta vacía o modelo «saturado»	Los modelos gratuitos a veces se saturan	Pulsar el nombre del modelo -> Seleccionar modelo -> elegir otro marcado como Gratis
Un modelo gratuito ha desaparecido de la lista	Los gratuitos son por tiempo limitado y cambian a menudo	Elegir otro marcado como Gratis y comprobar su política de datos en la documentación de opencode
No encuentra la carpeta en Añadir proyecto	El buscador no muestra carpetas ocultas o temporales	Escribir la ruta desde el usuario: Documents/practica_agente
El agente parece colgado o tarda mucho	Está trabajando, esperando un permiso o el modelo está saturado	Mirar si hay un aviso de «Permiso requerido» pendiente y si los pasos avanzan; si sigue igual, probar otro modelo gratuito o abrir una sesión nueva
Errores de conexión, certificados o "proxy"	Red corporativa o proxy que filtra la salida a internet	Probar con otra red o con el móvil como punto de acceso; pedir al informático la configuración del proxy
El firewall o el antivirus bloquean el programa	Bloqueo de ejecución o de conexión local	Añadir una excepción para Node.js y para la dirección local; si no se puede, usar otro portátil
"Permiso denegado" al leer o escribir una carpeta	Permisos del sistema operativo, carpeta sincronizada o de solo lectura	Usar una carpeta propia y sencilla del usuario; evitar carpetas sincronizadas en la nube
El agente dice que no puede escribir el archivo	Se ha pulsado Denegar, o la ruta está fuera de la carpeta del proyecto	Mover la tarea dentro de la carpeta de trabajo o repetir la petición y pulsar Permitir una vez tras leer el aviso

Síntoma	Causa probable	Solución
Un cambio en opencode.json, AGENTS.md o una skill no surte efecto	La configuración y las skills se leen al arrancar	Ctrl + C, opencode web y Nueva sesión
Las respuestas empeoran o se vuelven repetitivas	El contexto se ha llenado: la mesa está llena de papeles	Abrir una sesión nueva y guardar lo importante en AGENTS.md o en una skill
El alumno cierra la terminal y opencode se para	El servidor vive en esa terminal: cerrarla lo apaga	Abrir una terminal, entrar en la carpeta con cd y volver a arrancar con opencode web
Un alumno no puede instalar nada	Portátil corporativo o usuario restringido	Práctica por parejas con un compañero que sí lo tenga funcionando

Frases de rescate para el instructor

Sirven para decir en voz alta cuando algo falla delante de la clase.

1. "Esto es exactamente lo que veníamos a ver: por esto existe el «¿seguro?» antes de salir de la sala."
2. "Tranquilo, esto le pasa a la mitad de la sala. Vamos a mirarlo entre los dos."
3. "Fallar aquí no es un problema tuyo, es información: acabamos de aprender qué límite tiene el agente."
4. "Mientras arreglamos esto, fíjate en lo que ya ha ido bien: ha leído tu carpeta."
5. "Vamos a leer juntos el mensaje. Casi siempre dice lo que pasa."
6. "Si en dos minutos no sale, seguimos con el compañero y volvemos a esto después."
7. "El agente se equivoca y se corrige; la responsabilidad de revisar es nuestra."
8. "No hace falta saber programar para esto, solo hace falta leer lo que te pide."
9. "Guarda una captura del error: es un buen apunte para tu cuaderno."
10. "Vamos al plan B: lo hacemos dentro de la carpeta de trabajo y lo vemos funcionando."

Criterios de evaluación de las prácticas

Se evalúa el criterio, no la velocidad. Nadie tiene que terminar todas las prácticas.

Nivel	Qué se observa
No lo consigue	No arranca el harness o no llega a enviar una petición. No distingue el chat del agente. Necesita que el instructor haga clic por él.
Lo consigue con ayuda	Completa la práctica con pistas del instructor. Reconoce el aviso de permiso pero no explica qué lo provoca. Confunde la carpeta de trabajo con la carpeta de configuración.
Lo consigue solo	Arranca, abre su carpeta como proyecto, formula la petición y verifica el resultado. Explica carpeta de trabajo, permiso y memoria permanente con sus propias palabras. Sabe qué hacer cuando algo falla (por ejemplo, reiniciar tras cambiar la configuración o elegir otro modelo gratuito).

Nivel	Qué se observa
Va más allá	Cambia la política de permisos a propósito y explica el efecto. Guarda un procedimiento en AGENTS.md o como skill y lo prueba, tras reiniciar, en sesión nueva. Razona sobre mínimo privilegio y datos sensibles.

Qué mirar en cada práctica

- **Práctica 1:** ¿ha añadido el proyecto y elegido un modelo gratuito? ¿el índice menciona archivos reales de su carpeta?
- **Práctica 2:** ¿aparece el aviso y el agente espera? ¿el alumno lo lee antes de pulsar y sabe qué está aprobando? ¿ha probado **Denegar**?
- **Práctica 3:** ¿la misma tarea dentro de la carpeta no pide permiso de serie, y sí con `opencode.json` tras reiniciar? ¿sabe explicar la diferencia?
- **Práctica 4:** ¿`AGENTS.md` está en la raíz? ¿se respeta en una sesión nueva? ¿la skill funciona tras reiniciar?
- **Práctica 5:** ¿el alumno abre el archivo, cuenta y verifica? ¿distingue lo que el agente ha leído de lo que ha interpretado?

Qué NO debe hacer el instructor

1. **No aprobar permisos en el portátil de un alumno sin explicarlo.** Si lo haces tú, le enseñas a aprobar a ciegas. Lee el aviso en voz alta y deja que decida él.
2. **No prometer que el agente nunca se equivoca ni que la herramienta no cambiará.** opencode cambia muy rápido (versiones casi semanales), los modelos gratuitos son «por tiempo limitado» y puede haber cambios que rompan lo que ves en esta guía. La revisión humana es parte del método.
3. **No usar datos reales sensibles en las demostraciones.** Ni nóminas, ni datos de salud, ni credenciales, ni contratos. Gratis no es privado: lo que el agente lee se envía al proveedor del modelo, y con algunos modelos gratuitos (como Big Pickle) puede usarse para mejorarlo. Prepara una carpeta de ejemplo aburrida.
4. **No culpar al alumno delante del grupo.** Un "esto le pasa a muchos" desatasca más que un "te has saltado un paso".
5. **No improvisar con datos de alumnos en pantalla compartida.** Nada de abrir su carpeta personal, su correo o sus archivos por muy rápido que sea.
6. **No presentar la carpeta de trabajo y los permisos como una garantía.** opencode **no es un sandbox**: reduce el riesgo, no aísla el programa del resto del ordenador, y con la política de serie ejecuta órdenes dentro de la carpeta sin preguntar. No deben ser el único control de seguridad. Recomienda mínimo privilegio, máquina virtual o contenedor desechable, copias de seguridad y revisar plugins y comandos antes de permitirlos.
7. **No meter credenciales propias en el portátil de un alumno.** Ya no hace falta clave de API, pero sigue valiendo la regla: no iniciar sesión con cuentas propias ni dejar claves o credenciales tuyas en equipos de alumnos.